

## Keras: Questões de treino

Deep Learning (DEPAI) · IASEG/PMERJ · Aulas 2 (06/10) e 3 (07/10)

### Fáceis

#### K01.

Quantos parâmetros tem a rede abaixo?

```
modelo = keras.Sequential([
    keras.Input(shape=(6,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(8, activation="relu"),
    layers.Dense(3, activation="softmax"),
])
```

- a) 248
- b) 275
- c) 189
- d) 302

#### K02.

`X_train` é um array com formato (960, 4): 960 cédulas, cada uma com 4 variáveis. Qual deve ser a camada de entrada da rede?

- a) `keras.Input(shape=(960, 4))`
- b) `keras.Input(shape=(4,))`
- c) `keras.Input(shape=(960,))`
- d) `keras.Input(shape=(4))`

#### K03.

Uma rede classifica a natureza da ocorrência entre furto, roubo e outro, com última camada `Dense(3, activation="softmax")`. Depois de `previsoes = modelo.predict(X_test)`, o array `previsoes` tem formato (412, 3). Como obter a classe prevista (0, 1 ou 2) de cada ocorrência?

- a) `(previsoes[:, 0] > 0.5).astype(int)`
- b) `previsoes.argmax(axis=1)`
- c) `previsoes.sum(axis=1)`
- d) `previsoes.argmax(axis=0)`

#### K04.

Qual trecho dá uma estimativa honesta de como a rede vai se sair com cédulas novas?

- a) `modelo.fit(X_train, y_train)` e depois `modelo.evaluate(X_train, y_train)`.
- b) `modelo.fit(X_train, y_train)` e depois `modelo.evaluate(X_test, y_test)`.
- c) `modelo.fit(X, y)`, com todas as cédulas, e depois `modelo.evaluate(X_test, y_test)`.
- d) `modelo.fit(X_test, y_test)` e depois `modelo.evaluate(X_test, y_test)`.

#### Médias

#### K05.

Uma rede deve indicar qual de 5 delegacias recebe o caso. O vetor `y_train` tem formato `(2000,)`, com valores inteiros de 0 a 4. A última camada é `Dense(5, activation="softmax")`, e o `compile` usa `loss="categorical_crossentropy"`. O `fit` para com o erro:

```
ValueError: Arguments `target` and `output` must have the same shape.  
Received: target.shape=(32,), output.shape=(32, 5)
```

Qual é a causa, e como corrigir?

- a) A softmax só funciona com até 3 classes; trocar por `activation="sigmoid"`.
- b) A `categorical_crossentropy` espera o rótulo em one-hot, como `[0, 0, 1, 0, 0]`; com rótulos inteiros, usar `loss="sparse_categorical_crossentropy"`, ou converter os rótulos para one-hot e manter a perda.
- c) Os rótulos precisam estar como texto, como “delegacia 3”; converter os inteiros em texto.
- d) A última camada deve ter um neurônio só, porque o rótulo tem uma coluna só; trocar por `Dense(1)`.

#### K06.

Antes de treinar a rede das cédulas, é preciso padronizar as variáveis. Qual trecho está correto?

a)

```
padronizador = StandardScaler().fit(X)                                # X: todas as cédulas  
X_train = padronizador.transform(X_train)  
X_test = padronizador.transform(X_test)
```

b)

```
padronizador = StandardScaler().fit(X_train)  
X_train = padronizador.transform(X_train)  
X_test = padronizador.transform(X_test)
```

c)

```
X_train = StandardScaler().fit_transform(X_train)  
X_test = StandardScaler().fit_transform(X_test)
```

- d) Nenhum: a rede ajusta um peso para cada variável, e isso compensa as diferenças de escala; padronizar é desnecessário.

### K07.

Um colega rodou:

```
modelo.fit(X_train, y_train, batch_size=32, epochs=50,  
          validation_split=0.2)
```

e a saída começou com:

Epoch 1/50

750/750 - loss: 0.5120 - accuracy: 0.7713 - val\_loss: 0.4371 - val\_accuracy: 0.8102

Aproximadamente quantas linhas tem `X_train`?

- a) 750
- b) 24.000
- c) 30.000
- d) 37.500

### Difíceis

### K08.

No Exercício 4 da Aula 2 (cédulas), um aluno trocou a camada de saída por `Dense(1, activation="softmax")` e manteve `loss="binary_crossentropy"`. O treino roda sem erro, mas a saída do `fit` mostra, em todas as épocas, exatamente a mesma perda e `accuracy: 0.4448`. No treino, 44,48% das cédulas são falsas (classe 1). O que explica isso?

- a) Sobreajuste: a rede decorou as cédulas falsas; usar `EarlyStopping`.
- b) A softmax de um número só é sempre 1: a rede diz “falsa” para todas as cédulas e acerta só as falsas, 44,48%. Como a saída não muda quando os pesos mudam, não há inclinação, e o treino não sai do lugar. Trocar por `activation="sigmoid"`.
- c) O otimizador `sgd` tem passo pequeno demais; trocar por `"adam"` resolve.
- d) Falta capacidade: acrescentar uma camada oculta com ReLU resolve.

### K09.

Um grupo quis usar parada antecipada e escreveu:

```
parar = keras.callbacks.EarlyStopping(monitor="loss", patience=5,  
                                     restore_best_weights=True)  
modelo.fit(X_train, y_train, epochs=200, validation_split=0.2,  
          callbacks=[parar])
```

O treino rodou as 200 épocas. Pelo histórico, a perda de validação foi mínima na época 18 e subiu daí em diante, enquanto a perda de treino caiu até o fim. No teste, o resultado ficou pior que o esperado. O que aconteceu?

- a) `patience=5` é pequeno demais; com `patience=50`, o treino teria parado na época certa.
- b) O callback olha a perda de **treino** ("`loss`"), que continua caindo mesmo durante o sobreajuste. Ele nunca para, e o “melhor” peso é o da época de menor perda de treino, perto da última. Trocar para `monitor="val_loss"`.
- c) O `EarlyStopping` só funciona com o otimizador `sgd`.
- d) `restore_best_weights=True` obriga o treino a ir até a última época; basta retirar esse argumento.

## K10.

Para comparar duas taxas de aprendizado, um aluno roda, em sequência, no mesmo notebook:

```
keras.utils.set_random_seed(42)
modelo = keras.Sequential(...)          # monta a rede uma vez

modelo.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
               loss="binary_crossentropy", metrics=["accuracy"])
modelo.fit(X_train, y_train, epochs=20, validation_split=0.2)
# val_accuracy na última época: 0.90

modelo.compile(optimizer=keras.optimizers.Adam(learning_rate=0.01),
               loss="binary_crossentropy", metrics=["accuracy"])
modelo.fit(X_train, y_train, epochs=20, validation_split=0.2)
# val_accuracy na última época: 0.95
```

Ele conclui que a taxa 0,01 é melhor. Qual é o problema?

- a) Nenhum: o novo `compile` reinicia a rede, e as duas taxas partiram do mesmo ponto.
- b) O `compile` não reinicia os pesos: o segundo `fit` continuou de onde o primeiro parou, e a rede com taxa 0,01 teve, na prática, 40 épocas. Para comparar, montar a rede de novo, com a mesma semente, antes de cada taxa.
- c) Bastava chamar `keras.utils.set_random_seed(42)` de novo antes do segundo `compile`.
- d) Não é possível comparar taxas com o Adam, porque ele ajusta a taxa sozinho.